

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 12 · LECTURE NOTES

Derandomisation

Taking the randomness back out.

MODULE OUTCOMES

- Apply the method of conditional expectations.
- Explain limited independence and when it suffices.
- Explain expander graphs and their role.
- Explain pseudorandom generators at a working level.
- Decide whether derandomisation is worth attempting.

1 The method of conditional expectations

THE PROBABILISTIC PROOF says $E[\text{value}] = m/2$, so some assignment achieves at least $m/2$ (Module 07 section 1).

THE DERANDOMISATION: decide the variables one at a time. At each step compute the conditional expectation under each of the two choices, and take whichever is larger.

$$E[\text{value}] = (1/2) E[\text{value} \mid x=0] + (1/2) E[\text{value} \mid x=1]$$

So at least one of the two branches has conditional expectation at least as large as the current value. Take it. The conditional expectation never decreases.

After all variables are fixed, the "expectation" is the actual value of the assignment -- and it is at least the original $m/2$.

-> A DETERMINISTIC algorithm with the same guarantee.

FOR MAX-CUT this becomes: place each vertex on whichever side currently has fewer of its already-placed neighbours. A greedy algorithm, DERIVED rather than invented.

THE REQUIREMENT: you must be able to COMPUTE the conditional expectation efficiently. For MAX-CUT it is a count of neighbours, which is trivial. When the conditional expectation is itself intractable, the method fails -- and that is the usual obstacle.

WHY THIS IS THE METHOD TO TRY FIRST

It is mechanical. Given the probabilistic analysis, the derandomisation follows — no new idea is required, only the ability to compute a conditional expectation.

It preserves the guarantee exactly. The deterministic algorithm achieves at least what the randomised one achieved in expectation, which is the best outcome you could hope for from a derandomisation.

And it frequently produces an algorithm you would recognise. Greedy set cover, greedy MAX-CUT, and a number of standard approximation algorithms are conditional-expectation derandomisations of their probabilistic proofs — which means a surprising fraction of natural-looking greedy algorithms were derived rather than guessed, and the probabilistic proof explains *why* the greedy choice is the right one.

So when you have a probabilistic approximation algorithm, spend an hour on this. It may give you a deterministic version with the identical ratio and no seed to manage, which is frequently worth more than the randomised version's simplicity.

2 Limited independence

MOST ANALYSES NEED PAIRWISE INDEPENDENCE, NOT FULL INDEPENDENCE

n fully independent random bits cost n bits of randomness. n pairwise independent bits cost only $O(\log n)$ bits — generate them from a short seed by a linear map over a finite field, which is exactly the universal hashing construction of Module 04 §2.

And a great many analyses use only pairwise independence — anything bounded by Chebyshev does (Module 02 §1), including the hash-table chain-length analysis and the second-moment arguments of Module 06 §2.

So check what your analysis actually uses rather than what it assumed. If Chebyshev sufficed, $O(\log n)$ random bits suffice — and then you can enumerate all $2^{O(\log n)} = \text{poly}(n)$ possible seeds deterministically.

Which is a complete derandomisation: try every seed, take the best outcome, and the whole algorithm is deterministic and polynomial-time. And **k-wise independence for larger k costs $O(k \log n)$ bits and covers more analyses** — including weaker Chernoff-like bounds, which brings a good deal more within reach. **The enumerate-all-seeds move is what makes this a complete technique rather than an economy.**

Independence	Random bits needed	What it supports
Full independence	n bits.	Chernoff bounds, and every analysis.
Pairwise	$O(\log n)$.	Chebyshev bounds; universal hashing (Module 04 §2).

Independence	Random bits needed	What it supports
4-wise	$O(\log n)$.	Fourth-moment bounds; the AMS sketch's analysis (Module 05 §1).
5-wise	$O(\log n)$.	Linear probing's analysis (Module 04 §2's table) — which is why that result took so long.
k-wise	$O(k \log n)$.	Weaker Chernoff-like tail bounds , which covers a substantial fraction of the remaining analyses.

3 Expanders and pseudorandom generators

EXPANDER GRAPHS LET YOU REUSE RANDOMNESS

An expander is a sparse graph in which every small set of vertices has many neighbours outside it — equivalently, a graph with high conductance (Module 09 §2) and therefore a large spectral gap.

A random walk on an expander mixes in $O(\log n)$ steps, so k steps of such a walk give k nearly-independent samples using only $\log n + O(k)$ random bits — rather than the $k \log n$ bits that k independent samples would require.

Which derandomises error amplification: repetition normally costs k times the randomness of a single run, and an expander walk costs almost none of it (Module 02 §4's amplification, made randomness-efficient) — so you keep the exponentially small failure probability at a logarithmic randomness cost.

And a random graph is an expander with high probability (Module 06 §4) — so the existence proof is itself probabilistic, and the explicit constructions, which came decades later and are considerably harder, are what make the derandomisation actually effective. Which is a pleasing circularity: the probabilistic method proves that the object needed to remove randomness exists.

PSEUDORANDOM GENERATORS, AND THE CONDITIONAL RESULT

A pseudorandom generator stretches a short seed into a long string that no efficient test can distinguish from truly random (CSCE 637 Module 08 §3's definition).

If strong enough generators exist, every randomised polynomial-time algorithm can be derandomised — $BPP = P$ — by running the algorithm on the output of the generator for every short seed and taking the majority.

And strong enough generators follow from circuit lower bounds (CSCE 637 Module 09) — so hardness implies derandomisation, which is the counterintuitive connection that course develops at length.

So the theoretical answer is 'probably yes, conditionally' — and the practical answer is that §1 and §2 are what you will actually use, because they are unconditional, implementable, and frequently take an afternoon.

4 Should you derandomise

- **When reproducibility is required. A deterministic algorithm needs no seed management at all**, which is a real operational benefit in builds, in regulated environments, and in anything that must be audited (Module 13 §1).
- **When good randomness is unavailable. Embedded systems, early boot, and constrained environments** may simply not have an entropy source (Module 01 §4), and a deterministic algorithm sidesteps the problem rather than working around it.

- **When the conditional expectations are easy to compute. Then it is an hour's work and you lose nothing (§1)** — and you may end up with a recognisable greedy algorithm that is also faster.
- **And not when the randomised algorithm is simpler and the seed is manageable — which is the usual case**, and is the honest answer most of the time. A skip list that you can read beats a red-black tree that you cannot (Module 04 §4).
- **And never for the adversary use (Module 01 §1) — derandomising a hash function restores exactly the worst case that the randomness was there to defeat**, and an adversary who can read your code can then construct it. **Which is the important point: derandomisation is appropriate for the simplification and sampling uses of randomness, and is actively self-defeating for the adversary use.**

WHAT THIS MODULE ADDS TO CSCE 637'S VIEW

CSCE 637 Module 08 asked whether randomness can be removed from a *complexity class* — whether $BPP = P$ — and the answer there is conditional and unresolved.

This module removes it from *specific algorithms* — and the answer is frequently yes, unconditionally, in an afternoon.

So the two questions have different answers and are about different things, which is the point CSCE 637 Module 08 §4's closing callout flagged and which this module makes concrete.

And the practical version is the useful one: you will never derandomise BPP, and you may well derandomise your own algorithm — and the techniques for the second are §1 and §2, neither of which depends on any open question.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Motwani & Raghavan — chapter 5	Conditional expectations and limited independence — the reference for §1 and §2.
Vadhan — Pseudorandomness (free)	Free in full, and the reference for §3 — expanders, extractors, and generators, with the derandomisation applications.
Hoory, Linial & Wigderson — Expander Graphs and Their Applications (free)	§3's expanders , surveyed thoroughly and readably.
Luby & Wigderson — Pairwise Independence and Derandomization (free)	§2's technique , with the constructions and the enumerate-all-seeds argument.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Derandomise the MAX-CUT algorithm** by conditional expectations and confirm the resulting greedy rule.
- 2 **Verify the deterministic version achieves at least $m/2$** on random graphs.
- 3 **Derandomise MAX-3SAT's 7/8 algorithm** the same way.
- 4 **Find an algorithm where the conditional expectation is intractable** and say why the method fails.
- 5 **Construct pairwise independent bits** from a logarithmic seed and verify the pairwise property.
- 6 **Confirm they are not 3-wise independent.**
- 7 **Take an analysis that used Chebyshev** and verify it still works with pairwise independence only.
- 8 **Enumerate all seeds** for a small instance and confirm the derandomisation is complete.
- 9 **Generate a random regular graph** and estimate its spectral gap to confirm it is an expander.
- 10 **For your own randomised algorithm, decide whether to derandomise it**, using §4's five criteria.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the method of conditional expectations and derive the MAX-CUT greedy rule from it.
- 2 What does the method require, and when does it fail?
- 3 Why is it the method to try first?
- 4 Why does pairwise independence cost only $O(\log n)$ bits?
- 5 Which analyses does it support, and which does it not?
- 6 Explain the enumerate-all-seeds argument.
- 7 What is an expander, and what does a walk on one buy?
- 8 What is the conditional result about $BPP = P$?
- 9 Give five criteria for whether to derandomise.
- 10 Why should you never derandomise the adversary use?